SIMATS ENGINEERING

Assessment Tool 2: Industry Problem Solving Task — Answer Key

Course Name	Cloud Computing and Big Data Analytics
Course Code	CSA15
Course Outcome	CO4 — Analyze big data characteristics and challenges across domains including security and application aspects (BL4)
Weightage	20%
Bloom's Level	Articulation (Level 4)
Question Count	5
Total Marks	40

Q1. An e-commerce company receives clickstream, transaction, and review data from millions of users daily. Propose a big data analytics pipeline and explain how the 5Vs influence architecture design.

Answer

Proposed Pipeline:

- Data Ingestion: Use Apache Kafka / Flume to capture real-time clickstream events and transaction logs, and batch connectors (Sqoop) for structured order/inventory data from RDBMS.
- Storage: Store raw data in a distributed data lake (Hadoop HDFS or Amazon S3) using a schema-on-read approach so all data types (clicks, transactions, reviews) can coexist.
- Processing: Use Apache Spark for both batch analytics (daily sales trends) and Spark Streaming/Structured Streaming for real-time processing (live recommendations, cart-abandonment alerts).
- Storage for analytics: Load curated/aggregated data into a data warehouse (e.g., Snowflake/Redshift) or NoSQL store (Cassandra/MongoDB) for fast dashboard and application queries.
- Serving/Visualization: Use BI tools (Power BI/Tableau) and recommendation APIs to surface insights to business teams and the website in real time.

How the 5Vs influence architecture design:

- Volume: Millions of daily users demand horizontally scalable distributed storage (HDFS/S3) and cluster-based processing (Spark) rather than a single database server.
- Velocity: Clickstream data arrives continuously, requiring a streaming ingestion layer (Kafka) and near-real-time processing engine to support live personalization.
- Variety: Clickstream (semi-structured JSON), transactions (structured), and reviews (unstructured text) require a flexible data lake architecture rather than a single rigid schema.
- Veracity: Review data can be noisy or fraudulent (fake reviews, bot clicks), so the pipeline must include data-cleaning and validation stages before analysis.
- Value: The architecture must include an analytics/ML layer (recommendation engines, churn prediction) that converts raw events into actionable business value, not just storage.

Q2. A healthcare organization collects structured patient records, semi-structured insurance claims, and unstructured medical images. Design an approach to manage and secure this data ecosystem.

Answer

Data Management Approach:

- Structured data (patient demographics, lab results): Store in a relational database or structured tables within a data warehouse for reliable transactional queries.
- Semi-structured data (insurance claims in XML/JSON/EDI formats): Store in a NoSQL document database (e.g., MongoDB) that can handle varying claim formats without a rigid schema.
- Unstructured data (MRI/CT scans, clinical notes): Store in a distributed file system or object storage (HDFS/S3) with metadata catalogs (e.g., Apache Atlas) linking images to the corresponding patient record.
- Use a unified data lake architecture with a metadata layer so all three data types can be queried together for holistic patient analytics.

Security Design:

- Access control: Implement Role-Based Access Control (RBAC) so only authorized clinicians/staff can view specific categories of patient data (principle of least privilege).
- Encryption: Encrypt data at rest (AES-256) and in transit (TLS/SSL) for all three data categories, especially medical images and claims data.
- De-identification/anonymization: Mask or tokenize personally identifiable information (PII) when data is used for research or analytics to comply with HIPAA/data protection regulations.
- Audit logging: Maintain immutable audit trails of every access/modification to patient records for compliance and breach detection.
- Regulatory compliance: Align storage and processing design with healthcare regulations (HIPAA, or India's DPDP Act) governing consent, retention, and data sharing.

This layered approach ensures the organization can integrate diverse healthcare data for analytics while maintaining strict confidentiality, integrity, and regulatory compliance.

Q3. A telecom company wants to detect churn patterns using high-volume customer interaction data. Identify the big data challenges and recommend suitable analytics and integration strategies.

Answer

Key Big Data Challenges:

- Volume and velocity: Call detail records (CDRs), app usage logs, and customer-support interactions are generated continuously at very high volume, straining traditional storage/processing.
- Data variety and integration: Interaction data comes from multiple heterogeneous systems — billing, CRM, network logs, social media — making integration difficult.
- Data quality: Missing, duplicate, or inconsistent records across systems can distort churn predictions.
- Real-time requirement: Detecting churn signals (e.g., dropped calls, complaint spikes) early enough to intervene requires near real-time analytics, not just periodic batch reports.

Recommended Analytics and Integration Strategy:

- Integration layer: Use ETL/integration tools (Apache NiFi, Talend) to consolidate CDRs, CRM records, and support-ticket data into a centralized data lake.
- Distributed processing: Use Apache Spark for large-scale feature engineering (call frequency, complaint history, usage decline trends) across the consolidated dataset.
- Predictive modeling: Apply machine learning classification models (e.g., logistic regression, random forest, gradient boosting) trained on historical churn labels to score each customer's churn risk.
- Real-time monitoring: Use a streaming layer (Kafka + Spark Streaming) to flag high-risk behavior (e.g., sudden drop in usage) as it happens, triggering retention offers automatically.
- Visualization and action: Feed churn scores into a dashboard for the retention team and integrate with CRM so at-risk customers are proactively targeted with personalized offers.

This combination of robust integration and a hybrid batch + streaming analytics strategy allows the telecom company to detect churn patterns accurately and intervene before customers leave.

Q4. A smart city project generates sensor streams from traffic, air quality, and surveillance systems. Explain how distributed file systems and data integration tools can support large-scale analysis.

Answer

Role of Distributed File Systems:

- Scalable storage: HDFS or a cloud-based distributed object store (Amazon S3) can absorb continuous, high-volume sensor streams from thousands of traffic cameras, air-quality sensors, and surveillance feeds without a single storage bottleneck.
- Fault tolerance: Data replication across nodes ensures that even if individual storage nodes fail, critical city-monitoring data (e.g., surveillance footage) is not lost.
- Parallel processing support: Because sensor data is distributed across the cluster, frameworks like Spark can process traffic and air-quality analytics in parallel across nodes, enabling city-wide analysis within minutes rather than hours.
- Cost-effective long-term storage: Historical sensor data (for trend analysis, urban planning) can be stored cheaply at scale using commodity-hardware-based distributed storage.

Role of Data Integration Tools:

- Unified ingestion: Tools like Apache NiFi or Kafka Connect can ingest heterogeneous streams — traffic sensor readings, air-quality metrics, video surveillance metadata — into a common pipeline in real time.
- Format harmonization: Integration tools transform varied sensor data formats (protocol buffers, JSON telemetry, video metadata) into a consistent schema for downstream analytics.
- Cross-domain correlation: By integrating traffic and air-quality data together, the city can correlate congestion patterns with pollution spikes, enabling smarter traffic-signal and emission-control decisions.
- Real-time dashboards: Integrated, cleaned data feeds real-time dashboards for city control rooms, supporting immediate decisions such as rerouting traffic or issuing air-quality alerts.

Together, distributed file systems provide the scalable and resilient storage backbone, while integration tools unify and streamline the diverse sensor data streams — enabling the smart city to perform large-scale, cross-domain analysis efficiently.

Q5. A financial analytics firm must comply with strict privacy requirements while processing high-velocity transaction data. Design a secure big data solution and justify the controls chosen.

Answer

Proposed Secure Solution:

- Ingestion with in-transit protection: Use Kafka with TLS-encrypted channels to stream high-velocity transaction data securely from source systems into the processing pipeline.
- Tokenization/masking at ingestion: Sensitive fields (account numbers, card numbers, PII) are tokenized or masked immediately upon ingestion, so downstream analytics never handle raw sensitive values.
- Encrypted storage: Store processed data in a distributed store (HDFS/S3 or a cloud data warehouse) with AES-256 encryption at rest, and separate encryption keys managed via a dedicated Key Management Service (KMS).
- Fine-grained access control: Apply Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) so analysts only see the minimum data needed for their role (principle of least privilege).
- Real-time fraud/anomaly detection: Use Spark Streaming with machine learning models to flag suspicious transaction patterns as they occur, without exposing raw data to human reviewers unnecessarily.

- Audit and compliance logging: Maintain tamper-proof audit logs of all data access and processing activity to support regulatory audits (e.g., PCI-DSS, RBI/SEBI guidelines, GDPR where applicable).
- Data minimization and retention policy: Retain only data necessary for regulatory and business purposes, with automated purging of expired records to reduce exposure risk.

Justification of Controls:

Encryption (at rest and in transit) protects data confidentiality against interception and unauthorized storage access. Tokenization ensures that even if data is breached, sensitive financial identifiers remain unusable. RBAC/ABAC limits the blast radius of any single compromised account. Real-time anomaly detection balances the need for speed (high-velocity data) with security, catching fraud without slowing the pipeline. Audit logging and data minimization directly address regulatory compliance requirements, which is essential for a financial analytics firm operating under strict privacy laws.